

# Digital Image Processing

## Lab Assignment #1

**Marpina Sai Sriram**  
M.Tech (Research)  
Indian Institute of Science, Bengaluru

### 1 Problem 1: Color Space

#### 1.1 (a) Convert the Image to YCbCr Format

##### 1.1.1 Results

The given `fabric.png` image was converted from the RGB color space to the YCbCr color space. The converted image was separated into its three components: luminance ( $Y$ ), blue chrominance ( $Cb$ ), and red chrominance ( $Cr$ ).

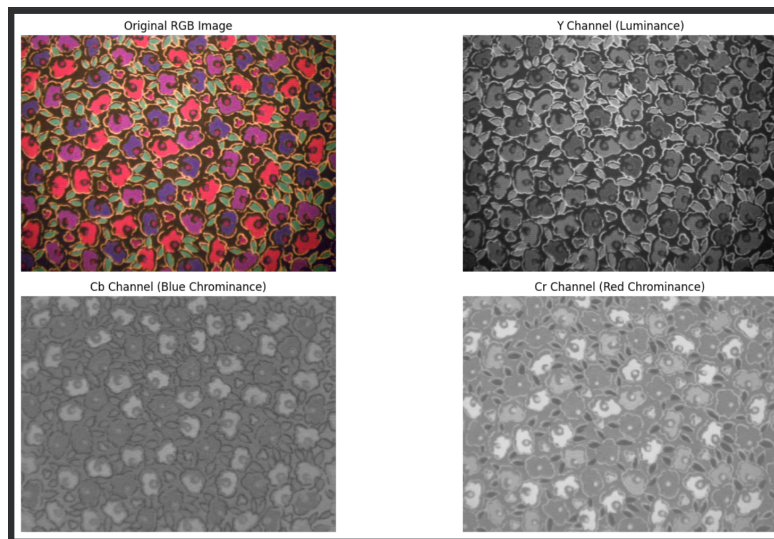


Figure 1: Original RGB image and the corresponding Y, Cb, and Cr channels after conversion to the YCbCr color space.

### 1.1.2 Inference

From the image, it can be clearly observed that the  $Y$  channel contains the luminance information and preserves the main structural details of the original image.

The regions corresponding to red-colored parts in the original RGB image appear brighter in the  $Cr$  channel. Similarly, regions corresponding to blue-colored parts in the original image appear brighter in the  $Cb$  channel.

Thus, the YCbCr color space separates brightness information into the  $Y$  channel and color-related information into the  $Cb$  and  $Cr$  channels.

## 1.2 (b) Compute the Per-Channel Variance of the YCbCr Image

### 1.2.1 Results

The variance of each channel in the YCbCr image was computed. The obtained values are shown below:

Table 1: Per-channel variance of the YCbCr image.

| Channel | Variance |
|---------|----------|
| $Y$     | 1658.15  |
| $Cb$    | 236.15   |
| $Cr$    | 542.22   |

### 1.2.2 Inference

From the computed variances, the  $Y$  channel has the highest variance, with a value of 1658.15. This indicates that the luminance values vary significantly across the image. This is expected because the image contains regions with different brightness levels, including dark boundaries and brighter colored regions.

The  $Cr$  channel has a variance of 542.22, which is higher than the variance of the  $Cb$  channel, 236.15. This indicates that there is more variation in the red chrominance component than in the blue chrominance component in the given image.

Thus, for this image, the variation in the three channels follows:

$$\text{Var}(Y) > \text{Var}(Cr) > \text{Var}(Cb).$$

Therefore, the luminance component has the largest variation, followed by the red chrominance component, while the blue chrominance component has the smallest variation.

### 1.3 (c) Chroma Sub-sampling

#### 1.3.1 Method

Chroma sub-sampling was performed on the  $C_b$  and  $C_r$  channels of the YCbCr image. For each  $2 \times 2$  block in the  $C_b$  and  $C_r$  channels, the average of the four values was computed, and all four elements in the block were replaced by this average.

For a  $2 \times 2$  block,

$$\begin{bmatrix} x_{11} & x_{12} \\ x_{21} & x_{22} \end{bmatrix},$$

the average value is given by

$$\bar{x} = \frac{x_{11} + x_{12} + x_{21} + x_{22}}{4}.$$

The block is then replaced by

$$\begin{bmatrix} \bar{x} & \bar{x} \\ \bar{x} & \bar{x} \end{bmatrix}.$$

This operation was applied independently to every  $2 \times 2$  block in both the  $C_b$  and  $C_r$  channels, while the luminance channel  $Y$  was left unchanged.

#### 1.3.2 Results

The modified  $C_b$  and  $C_r$  channels were combined with the original  $Y$  channel to form the modified YCbCr image. The modified YCbCr image was then converted back to RGB for visual comparison with the original image.

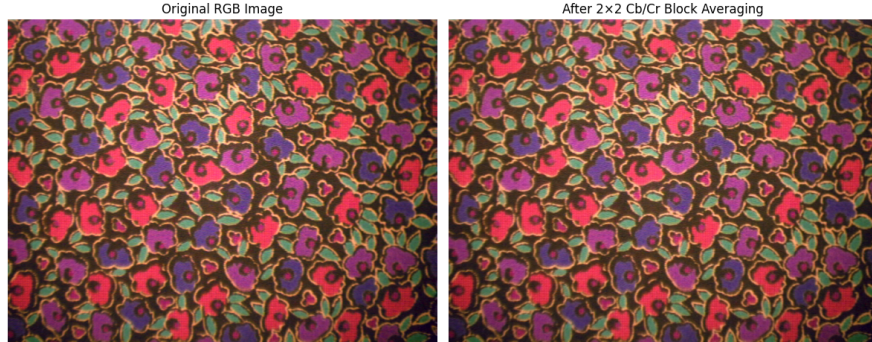


Figure 2: Comparison of the original RGB image and the RGB image reconstructed from the YCbCr image after  $2 \times 2$  block averaging of the  $C_b$  and  $C_r$  channels.

### 1.3.3 Inference

From the results, the RGB image obtained after  $2 \times 2$  averaging of the  $C_b$  and  $C_r$  channels appears very similar to the original RGB image. Only minor changes in color details can be observed, while the overall structure and brightness of the image are well preserved.

This occurs because the  $Y$  channel, which contains the luminance (brightness) information, is kept unchanged. The averaging operation only reduces the spatial variation of the chrominance components  $C_b$  and  $C_r$ .

Therefore, the experiment demonstrates that chrominance information can be reduced through sub-sampling while preserving most of the visually important information in the image.

## 1.4 (d) Artifacts Due to Chroma Sub-sampling

Chroma sub-sampling is expected to produce artifacts mainly near **sharp color boundaries and high-frequency color variations**. When a  $2 \times 2$  block contains pixels belonging to different colored regions, averaging the  $C_b$  and  $C_r$  values produces an intermediate chrominance value. After converting back to RGB, this can result in color bleeding or incorrect colors near the boundary.

Therefore, artifacts are more likely to occur around sharp color transitions, thin colored lines, text, and fine color patterns. In smooth regions, the chrominance values vary slowly, so averaging has little visible effect.

Hence, the intuition is that:

Large chrominance variation within a block  $\Rightarrow$  greater chance of visible artifacts

while smooth chrominance regions are less affected by sub-sampling.

## 2 Problem 2: Histogram Computation

### 2.1 (a) Histogram of coins.png

#### 2.1.1 Method

The image `coins.png` is a grayscale image quantized to 8 bits. Therefore, each pixel can have an intensity value in the range 0 to 255, giving a total of 256 possible intensity levels.

The histogram of the image was computed by finding the frequency of pixels corresponding to each intensity level. Mathematically, the histogram  $H(k)$  is defined as

$$H(k) = \sum_{i=1}^M \sum_{j=1}^N I[I(i, j) = k], \quad k = 0, 1, \dots, 255,$$

where  $I(i, j)$  represents the intensity of the pixel at position  $(i, j)$ , and  $I[\cdot]$  is an indicator function that is equal to 1 when the pixel intensity is  $k$  and 0 otherwise.

Thus,  $H(k)$  represents the number of pixels having intensity value  $k$ .

### 2.1.2 Results

The histogram of the `coins.png` image is shown below.

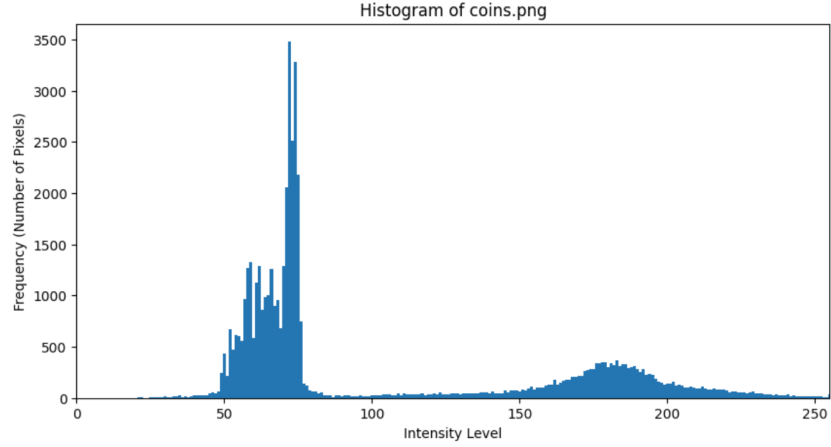


Figure 3: Histogram of the grayscale image `coins.png`.

### 2.1.3 Observation

The histogram shows the distribution of grayscale intensity values in the `coins.png` image. The intensity values are distributed over a range of gray levels corresponding to the dark background, coin regions, and brighter portions of the coins.

The peaks in the histogram indicate intensity levels that occur frequently, whereas lower values indicate intensity levels that occur less frequently. Therefore, the histogram provides useful information about the overall brightness and contrast of the image.

Since the image contains both relatively dark and bright regions, the histogram covers a considerable range of intensity values. This indicates that the image contains significant variation in pixel intensities.

## 2.2 (b) Expectation and Variance of Intensity

The expectation and variance of the pixel intensity were computed using the histogram and then verified using the actual pixel values.

Let  $H(k)$  denote the histogram frequency at intensity level  $k$  and let  $N$  be the total number of pixels. The probability of intensity level  $k$  is

$$P(k) = \frac{H(k)}{N}.$$

The expectation (mean) of the intensity is given by

$$\mu = E[X] = \sum_{k=0}^{255} kP(k).$$

The variance is calculated as

$$\sigma^2 = \sum_{k=0}^{255} (k - \mu)^2 P(k).$$

The results obtained using the histogram were then compared with the results obtained directly from the pixel values.

Table 2: Comparison of expectation and variance obtained using the histogram and actual pixel values.

| Method              | Expectation | Variance    |
|---------------------|-------------|-------------|
| Histogram           | 103.305002  | 3130.804221 |
| Actual Pixel Values | 103.305002  | 3130.804221 |

### 2.2.1 Verification

The expectation obtained using the histogram is

$$\mu = 103.305002$$

and the corresponding variance is

$$\sigma^2 = 3130.804221.$$

When these values were calculated directly from the actual pixel intensities, the results were also

$$\mu = 103.305002, \quad \sigma^2 = 3130.804221.$$

The differences between the two methods are

$$|\mu_{\text{hist}} - \mu_{\text{actual}}| = 0,$$

and

$$|\sigma_{\text{hist}}^2 - \sigma_{\text{actual}}^2| = 0.$$

Thus, the histogram-based computation is exactly verified by the calculation using the actual pixel values.

### 2.2.2 Observation

The expectation of approximately 103.31 represents the average brightness of the image. The variance of approximately 3130.80 indicates the amount of variation in intensity values around the mean. The exact agreement between the histogram-based and direct pixel-based calculations confirms that the histogram correctly represents the intensity distribution of the image.

## 2.3 (c) Quantization to 6 Bits

An 8-bit grayscale image has  $2^8 = 256$  possible intensity levels, ranging from 0 to 255. When the image is quantized to 6 bits, the number of possible intensity levels becomes

$$2^6 = 64.$$

Therefore, the 256 intensity levels of the original histogram need to be mapped into 64 intensity levels.

The 8-bit intensity value  $k$  can be mapped to a 6-bit intensity value using

$$k_6 = \left\lfloor \frac{k}{4} \right\rfloor, \quad k = 0, 1, \dots, 255.$$

Thus, every four consecutive intensity levels in the original image are mapped to one 6-bit intensity level:

$$\{0, 1, 2, 3\} \rightarrow 0,$$

$$\{4, 5, 6, 7\} \rightarrow 1,$$

...

$$\{252, 253, 254, 255\} \rightarrow 63.$$

Hence, the histogram of the 6-bit image can be computed directly from the original 8-bit histogram. The frequency of the  $r$ -th 6-bit intensity level is

$$H_6(r) = \sum_{k=4r}^{4r+3} H_8(k), \quad r = 0, 1, \dots, 63,$$

where  $H_8(k)$  is the original 8-bit histogram.

### 2.3.1 Results

The histogram after quantization to 6 bits is shown below.

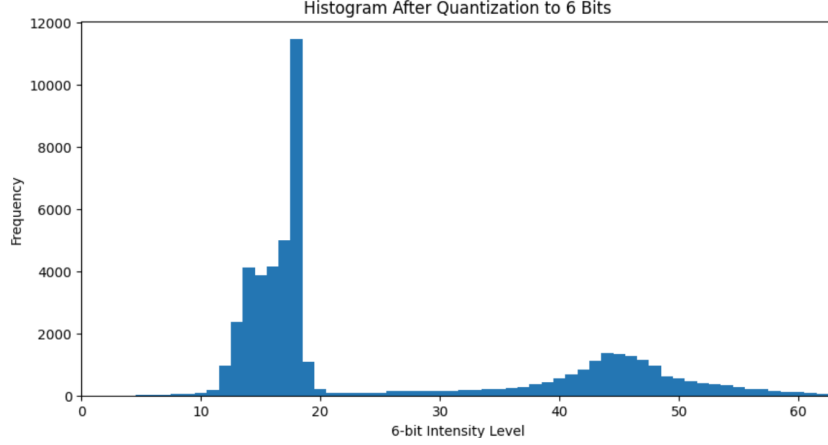


Figure 4: Histogram of `coins.png` after quantization to 6 bits.

### 2.3.2 Observation

The 6-bit histogram contains only 64 intensity levels instead of the 256 levels present in the original 8-bit histogram. Each bin in the 6-bit histogram combines the frequencies of four consecutive bins from the original 8-bit histogram.

It is therefore not necessary to first quantize every pixel of the image and then compute its histogram. Since quantization simply groups the original intensity levels, the new histogram can be obtained directly by combining the corresponding bins of the original histogram.

Thus,

$$H_6(r) = H_8(4r) + H_8(4r + 1) + H_8(4r + 2) + H_8(4r + 3)$$

This approach produces the same histogram while avoiding the additional step of creating and storing a separate quantized image.

## 3 Problem 3: Otsu's Binarization

### 3.1 (a) Binarization of `coins.png` using Otsu's Method

The grayscale image `coins.png` was binarized using Otsu's thresholding method. The optimal threshold  $t^*$  was determined by minimizing the within-class variance  $\sigma_{\text{Intra}}^2(t)$ .

For a given threshold  $t$ , the image is divided into two classes:

$$C_0 = \{x \mid x \leq t\},$$

$$C_1 = \{x \mid x > t\}.$$

Let  $\omega_0(t)$  and  $\omega_1(t)$  denote the probabilities of the two classes, and let  $\sigma_0^2(t)$  and  $\sigma_1^2(t)$  denote their corresponding variances. The within-class variance is given by

$$\sigma_{\text{Intra}}^2(t) = \omega_0(t)\sigma_0^2(t) + \omega_1(t)\sigma_1^2(t).$$

The optimal threshold is the threshold that minimizes the within-class variance:

$$t^* = \arg \min_{t \in \{0, \dots, 255\}} \sigma_{\text{Intra}}^2(t)$$

The within-class variance was evaluated for all possible threshold values from 0 to 255.

### 3.1.1 Results

The minimum within-class variance was obtained at the optimal threshold

$$t^* = 125$$

with the corresponding minimum within-class variance

$$\sigma_{\text{Intra}}^2(125) = 265.1024571148033$$

Figure 5 shows the within-class variance for different threshold values. The vertical dashed line indicates the optimal threshold  $t^* = 125$ .

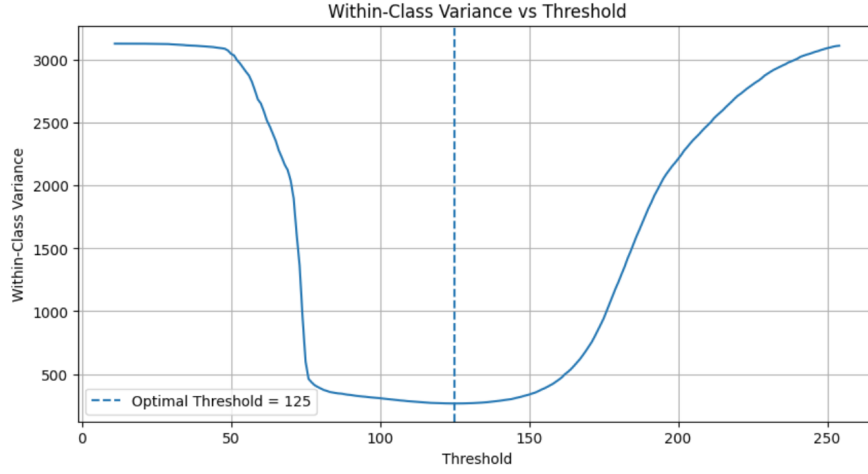


Figure 5: Within-class variance as a function of threshold. The minimum occurs at the optimal threshold  $t^* = 125$ .

Using the optimal threshold  $t^* = 125$ , the image was binarized according to

$$B(i, j) = \begin{cases} 255, & I(i, j) > 125, \\ 0, & I(i, j) \leq 125. \end{cases}$$

The original image and the resulting binary image are shown in Figure 6.

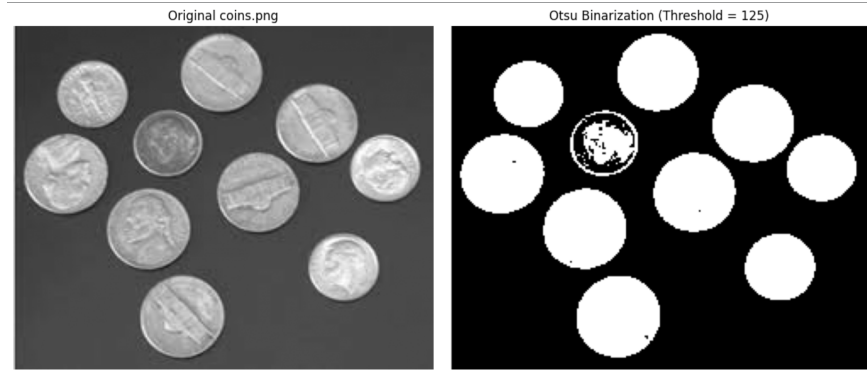


Figure 6: Original `coins.png` and its binarized image using Otsu's optimal threshold  $t^* = 125$ .

### 3.1.2 Observation

From Figure 5, the within-class variance reaches its minimum at the threshold  $t = 125$ . Therefore, Otsu's method selects 125 as the optimal threshold.

The resulting binary image provides a clear separation between the brighter coin regions and the darker background. Thus, minimizing the within-class variance provides an effective threshold for separating the foreground and background in `coins.png`.

### 3.2 (b) Intensity Inversion and Otsu's Threshold

Each pixel intensity  $p_i$  of the original image was inverted using the transformation

$$p'_i = 255 - p_i.$$

Thus, dark pixels in the original image become bright pixels in the new image, and bright pixels become dark.

Otsu's thresholding method was then applied to the intensity-inverted image to determine the new optimal threshold.

#### 3.2.1 Results

For the original image, the optimal Otsu threshold was

$$t^* = 125.$$

After intensity inversion, the corresponding threshold can be determined from

$$p'_i = 255 - p_i.$$

Because the thresholding convention used is

$$I(i, j) \leq t \quad \text{for Class 0}$$

and

$$I(i, j) > t \quad \text{for Class 1,}$$

the new threshold is

$$t_{\text{new}} = 255 - t^* - 1.$$

Therefore,

$$t_{\text{new}} = 255 - 125 - 1 = 129.$$

Hence, the expected new Otsu threshold is

$$\boxed{t_{\text{new}} = 129}.$$

Otsu's method was applied directly to the inverted image, and the computed threshold was also found to be

$$t_{\text{Otsu}} = 129.$$

Thus, the result is verified.

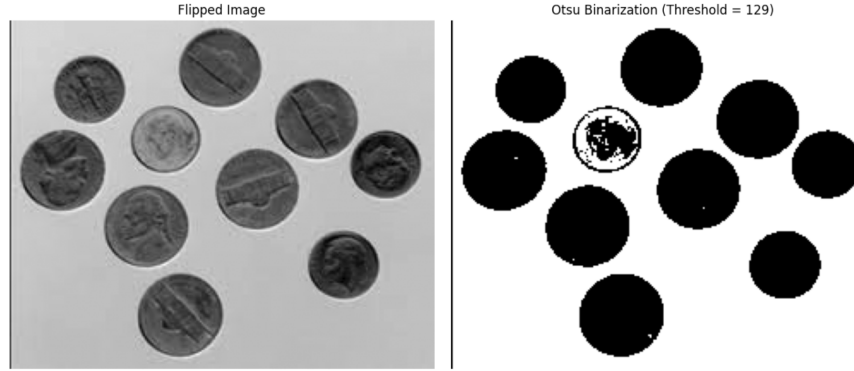


Figure 7: Intensity-flipped `coins.png` and its Otsu binarization using the new optimal threshold  $t^* = 129$ .

### 3.2.2 Observation

### 3.2.3 Verification

The expected and computed thresholds are compared below:

| Quantity                           | Threshold |
|------------------------------------|-----------|
| Original image                     | 125       |
| Expected threshold after inversion | 129       |
| Computed Otsu threshold            | 129       |

Since the expected and computed thresholds are identical, the result is verified.

### 3.2.4 Observation

Intensity inversion reverses the ordering of the gray levels. Therefore, the threshold also shifts to the corresponding location in the inverted intensity range. The original threshold of 125 maps to a new threshold of 129. Importantly, the segmentation of the image remains equivalent up to the reversal of the binary intensity values.

## 3.3 (c) Adding a Constant Positive Offset

Instead of flipping the pixel intensities, a constant positive offset of 20 was added to every pixel of the original image. Since the image is an 8-bit grayscale

image, the pixel values were clipped to a maximum of 255 whenever the addition exceeded this value.

The modified pixel intensity is given by

$$p'_i = \min(p_i + 20, 255).$$

Thus, pixels with intensity values less than or equal to 235 are increased by 20, while values greater than 235 are clipped to 255.

### 3.3.1 Finding the New Otsu Threshold

For the original image, the optimal Otsu threshold was

$$t_{\text{old}} = 125.$$

If the pixel values are shifted by 20 without clipping, the threshold is expected to shift by the same amount:

$$t_{\text{expected}} = t_{\text{old}} + 20.$$

Therefore,

$$t_{\text{expected}} = 125 + 20 = 145.$$

Otsu's method was then applied to the modified image to determine the actual threshold. The computed result was

$$t_{\text{new}} = 145.$$

Hence, the threshold changed from 125 to 145, giving a threshold shift of

$$\Delta t = t_{\text{new}} - t_{\text{old}} = 145 - 125 = 20.$$

### 3.3.2 Within-Class Variance

For the original image, the minimum within-class variance was

$$\sigma_{\text{Intra,old}}^2 = 265.1024571148033.$$

After adding the offset of 20 and clipping the pixel values, the minimum within-class variance became

$$\sigma_{\text{Intra,new}}^2 = 255.93054363922528.$$

The comparison is given below:

| Image        | Otsu Threshold | Minimum Within-Class Variance |
|--------------|----------------|-------------------------------|
| Original     | 125            | 265.1024571148033             |
| Offset (+20) | 145            | 255.93054363922528            |

### 3.3.3 Binarization of the Modified Image

The modified image was binarized using the new Otsu threshold  $t_{\text{new}} = 145$ . The binarization rule is

$$B(i, j) = \begin{cases} 255, & I'(i, j) > 145, \\ 0, & I'(i, j) \leq 145. \end{cases}$$

The resulting binarized image is shown in Figure 8.

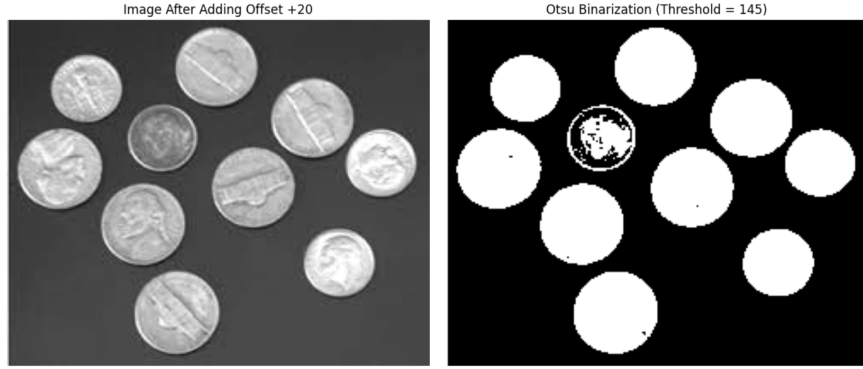


Figure 8: Image after adding an intensity offset of 20 and its Otsu binarization using the new threshold  $t_{\text{new}} = 145$ .

### 3.3.4 Analysis and Observation

Adding a constant positive offset shifts the intensity distribution towards higher intensity values. Consequently, the Otsu threshold also shifted towards a higher intensity value.

The original threshold was

$$125,$$

while the new threshold after adding 20 was

$$145.$$

Thus,

$$\boxed{125 \rightarrow 145}$$

and the threshold increased by exactly 20.

Although pixel values exceeding 255 were clipped to 255, the clipping did not alter the location of the optimal Otsu threshold in this experiment. The minimum within-class variance changed from 265.1024571148033 to 255.93054363922528 because clipping modifies the intensity distribution, even though the optimal threshold remained shifted by exactly 20.

Therefore, this experiment demonstrates that adding a constant intensity offset generally shifts the Otsu threshold by the same amount, provided that clipping does not significantly affect the optimal separation between the two intensity classes.

## 4 Problem 4: Connected Components

### 4.1 Binarization of `quote.png`

The image `quote.png` was first converted to a grayscale image. Otsu's thresholding method was then used to binarize the image. Since the characters in the image are black and the background is white, inverse binary thresholding was used so that the character pixels become the foreground pixels.

The transformation used was

$$B(i, j) = \begin{cases} 255, & I(i, j) < t, \\ 0, & I(i, j) \geq t, \end{cases}$$

where  $t$  is the Otsu threshold.

The Otsu threshold obtained for `quote.png` was

$$\boxed{t = 142}.$$

Thus, the black character pixels were converted to 255 and the background pixels were converted to 0.

Binarized Image (Otsu Threshold = 142)

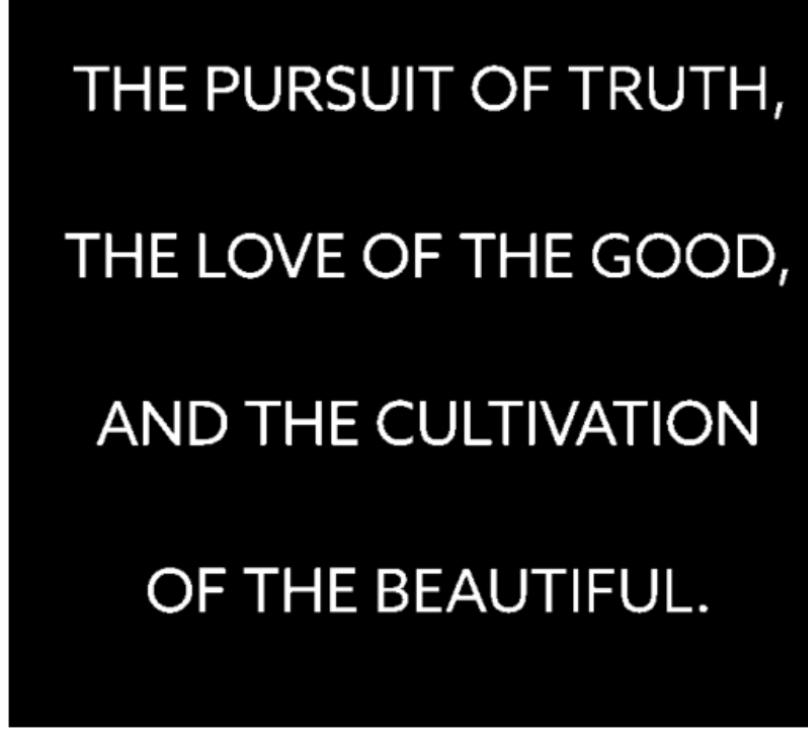


Figure 9: Binarized `quote.png` using Otsu's thresholding.

## 4.2 Manual Connected Component Analysis

Connected component analysis was performed manually using a Depth-First Search (DFS) based flood-fill algorithm. The built-in connected component functions were not used.

An 8-connected neighborhood was considered for every foreground pixel. The eight neighboring positions are

$$\mathcal{N}_8 = \{(-1, -1), (-1, 0), (-1, 1), (0, -1), (0, 1), (1, -1), (1, 0), (1, 1)\}.$$

For every unlabelled foreground pixel, a new component label was assigned. A DFS was then performed to find all foreground pixels connected to that pixel.

The algorithm scanned the complete image and assigned a unique label to each connected component. The total number of connected components obtained was

67.

The area of each component was calculated by counting the number of pixels belonging to that component.



Figure 10: Connected components obtained using 8-connected neighborhood. Each component is assigned a different label.

### 4.3 Largest Connected Component

After obtaining all connected components, the number of pixels in each component was compared to identify the largest component.

The largest connected component was found to be

Component 52

with an area of

546 pixels.

Thus, Component 52 was selected as the largest connected component corresponding to a character.

| Property                       | Value      |
|--------------------------------|------------|
| Number of connected components | 67         |
| Largest component label        | 52         |
| Largest component area         | 546 pixels |

#### 4.4 Highlighting the Largest Component

The pixels belonging to Component 52 were identified from the component-label image. These pixels were then colored red while keeping all other pixels unchanged from the original grayscale image.

The RGB value used for the selected component was

$$(255, 0, 0),$$

which corresponds to red.

The resulting modified image is shown in Figure 11.

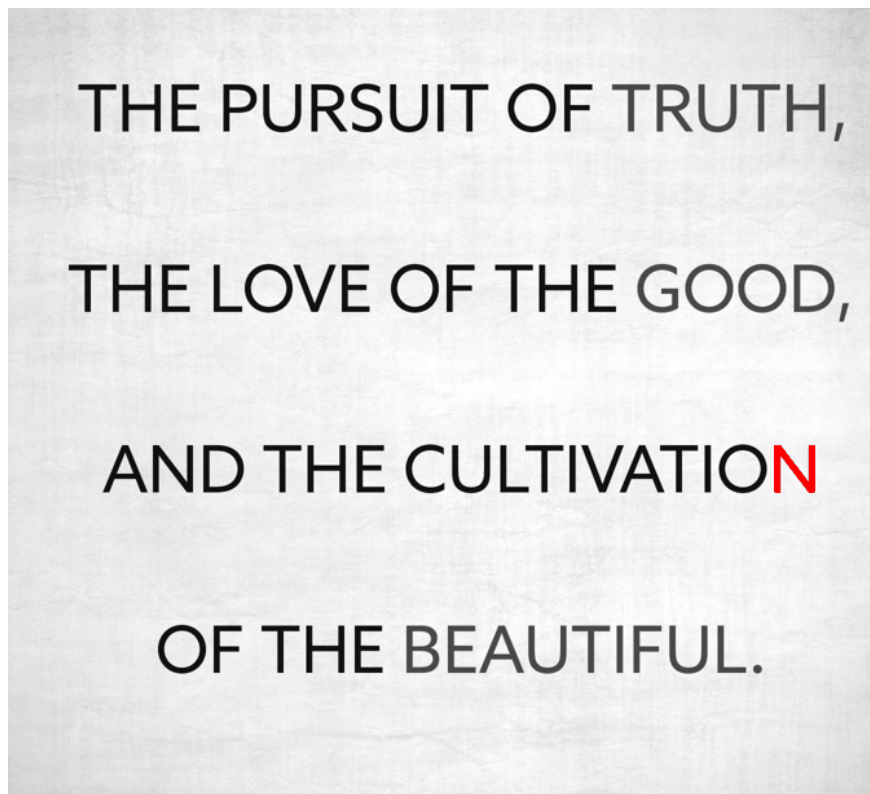


Figure 11: Original image with the largest connected component (Component 52) highlighted in red. The component contains 546 pixels.

## 4.5 Observation

The binary image was successfully segmented into 67 connected components using 8-connectedness. Each connected component represents a group of foreground character pixels that are connected either horizontally, vertically, or diagonally.

By comparing the number of pixels in each component, Component 52 was identified as the largest component, containing 546 pixels. This component was then highlighted in red on the original image. The result demonstrates that connected component analysis can be used to separate and identify individual character regions in a binary image.